Infosys Springboard Virtual Internship 6.0: Milestone 1 Report

Project Name: Crypto Volatility and Risk Analyzer

Intern Name: Durga Prasad Bhogavilli

Batch: 11 | **Team:** C

Mentor: Sangeetha Mahalingam

1. Introduction

The primary objective of this training program was to establish a core understanding of Python as a development environment. The sessions were designed to be interactive, utilizing notebooks that seamlessly integrate executable code, output visualization, and documentation. To align with industry standards, the curriculum also introduced version control systems like GitHub and the fundamentals of the Agile development process.

2. Concepts Covered

During the sessions, the mentor highlighted the utility of various tools for interactive Python development. We explored the specific structure of `.ipynb` files, distinguishing between code cells for execution and markdown cells for documentation.

Key technical concepts included:

- **Core Fundamentals:** Writing output statements, capturing user input, variable assignment, and handling data types like integers, floats, and strings.
- **Data Manipulation:** Techniques for string manipulation, numeric calculations, and formatting were demonstrated through hands-on examples.
- **Process & Tools:** The sessions covered using Markdown for code logic documentation, managing repositories and commits via GitHub, and understanding Agile components such as sprints and iterative development.

3. Technical Implementation Overview

During the training sessions, I wrote and executed several Python scripts to practice the concepts discussed. Below is a summary of the key implementations, complete with code snippets and their execution results.

A. Basic Input and Output

I started by writing simple scripts to handle input and output operations. One of the first tasks involved displaying a static "Hi..!" greeting. I then extended this logic to prompt the user for their name, capturing the input "Dp" and printing a personalized greeting.

Code Snippet:

```
print("Hi..!")
name = input('Enter your Name: ')
print('Hi ', name)
```

Output:

```
Hi..!
Enter your Name: Dp
Hi  Dp
```

B. Data Types and Calculations

To understand data types, I implemented arithmetic operations using both integers and floating-point numbers. I wrote a script that accepted two integer inputs (A and B) and calculated their sum. Additionally, I created a program to calculate the area of a triangle by accepting the base and height as float values.

Code Snippet:

```
# Integer Input
a = int(input('Enter the value for A: '))
b = int(input('Enter the value for B: '))
c = a+b
print(c)

# Area of Triangle
b = float(input("Enter the Base Length of Triangle: "))
h = float(input("Enter the Height Length of Triangle: "))
a = 0.5 * b * h
print("Area = ", a)
```

Output:

```
106
Area =  31.5
```

C. Modules and Libraries

I explored Python's standard library capabilities by importing the `calendar` module. I wrote a script that asked the user to input a specific year and month number to generate a formatted calendar.

Code Snippet:

```
import calendar
```

```
year = int(input("Please Enter the year Number: "))
month = int(input("Please Enter the month Number: "))

print(calendar.month(year, month))
```

Output:

```
     June 2005
Mo Tu We Th Fr Sa Su
       1  2  3  4  5
 6  7  8  9 10 11 12
13 14 15 16 17 18 19
20 21 22 23 24 25 26
27 28 29 30
```

D. String Manipulation and Methods

I performed extensive exercises on string manipulation. This included iterating through the string "MISSISSIPPI" to print each character on a new line. I also applied various string methods to the sentence "life iS beautiful !!!" to convert case, replace characters, and split the string.

Code Snippet:

```
# Looping through string
for x in 'MISSISSIPPI':
    print(x)

# String Methods
s = "life iS beautiful !!! "
print(s.upper())
print(s.replace("l",'w'))
print(s.split("i"))
```

Output:

```
M
I
S
S
I
S
S
I
P
P
I
LIFE IS BEAUTIFUL !!!
wife iS beautifuw !!!
['l', 'fe ', 'S beaut', 'ful !!! ']
```

E. String Slicing and Concatenation

I practiced accessing specific parts of strings using slicing techniques on the string "Python Programming". I also demonstrated string concatenation by joining two separate phrases into a single cohesive sentence.

Code Snippet:

```
str = "Python Programming"
# Slicing from 1st to 4th index
print(str[1:5])
# Slicing from 0th to 2nd index
print(str[:3])

a = 'life is beautiful !!!'
b = 'Live happily'
print("Concatenation: " + a + b)
```

Output:

```
ytho
Pyt
Concatenation: life is beautiful !!!Live happily
```

F. String Formatting

To practice dynamic text generation, I utilized the `.format()` method. By using positional arguments, I injected variables into a sentence structure.

Code Snippet:

```
# Positional arguments in formatting
print('{1} and {0} are best friends'.format('Dp','Krishna'))
```

Output:

```
Krishna and Dp are best friends
```

G. Control Flow (Conditional Statements)

I implemented decision-making logic using `if-else` statements. One script determined whether a user-inputted number was even or odd, while another compared three different integers (A, B, and C) to find the largest value.

Code Snippet:

```
# Even or Odd Check
num = int(input('Enter the number : '))

if num % 2 == 0:
  print('even: ', num)
else:
  print('odd: ',num)
```

```
# Greatest of three numbers
a = int(input('Enter A: '))
b = int(input('Enter B: '))
c = int(input('Enter C: '))

if (c>a and c>b):
  print('C is bigger')
```

Output:

```
odd:  27
C is bigger
```

H. Logical Operations

Finally, I applied logical operators to validate data. I wrote a guessing game that checked if a number fell within a specific range, and a membership check that verified if a specific register number existed within a pre-defined list.

Code Snippet:

```
# Range check
num = int(input("Guess a Number: "))
if num>2 and num<79:
    print("Correct Guess!")
else:
    print("Incorrect Guess!")

# List membership
regno = int(input("Enter Register no : "))
if regno in [98,101,105,109,110]:
    print("pass")
else:
    print("Fail")
```

Output:

```
Incorrect Guess!
Fail
```

4. Activities & Learning Outcomes

Throughout the training, I was actively engaged in writing and executing Python programs within Jupyter Notebooks, analyzing outputs, and refining code to solidify my understanding. I utilized markdown cells effectively to document code functionality, ensuring the notebooks were well-structured.

Key Takeaways:

- **Python Proficiency:** I gained a robust grasp of Python basics, including input-output operations, variable handling, and logic construction.
- **Workflow Management:** I learned to manage and share .ipynb files using GitHub, enhancing my ability to work collaboratively.
- **Methodology:** The training improved my logical problem-solving skills and provided a practical understanding of Agile development practices.

5. Conclusion

The Milestone 1 sessions successfully demonstrated the practical implementation of Python programming concepts. By integrating GitHub for version control and introducing Agile methodologies, the training provided valuable insights into real-world software development cycles. These skills form a strong foundation for my future academic and professional projects.